

Calcul Parallèle

TP : Partie 2

Thomas STAVIS
(thomas.stavis@ens-lyon.fr)

Jonathan Rouzaud-Cornabas
(jonathan.rouzaud-cornabas@inria.fr)

Rendu Même consigne que pour la partie 1 du TP1.

Notation La partie 2 de ce TP est sur 28 points. La note de cette partie sera de *minimum*(20, votre note à la partie)/20. La note finale du TP1 sera une moyenne des notes des 2 parties.

2 Multiplication de matrices (/28)

Dans cette section nous allons utiliser le parallélisme pour augmenter la vitesse d'exécution, et donc le speed-up, de la multiplication de matrices. Une importance particulière sera donnée au raisonnement (les erreurs de calcul, tant que le raisonnement est juste et le résultat non-aberrant, ne sont pas pénalisées).

2.1 Multi-parallélisme (/14)

Dans la partie 1 de ce TP nous avons abordé le multi-threading, mais cette technique est limitée (il n'y a pas une infinité de threads dans un ordinateur). Cependant, il existe d'autres types de parallélismes qu'on peut utiliser en même temps pour améliorer le speed-up (on n'a pas beaucoup de temps dans ces TPs, mais je vous conseille de chercher ce que sont ces outils par vous même) :

- instructions AVX (CPU)
- pipelining
- multi-cœurs physiques
- multi-processus
- multi/hyper-threading
- GPU
- multi-CPU/GPU
- multi-ordinateurs

Important : Pour les besoins du TP, toutes les matrices seront carrées et de taille 2^m avec $m \in \mathbb{N}$ (donc comportent $(2^m)^2$ éléments), et contiennent des nombres entiers inférieurs à 2^{16} et codés sur 64 bits. Voici l'algorithme "classique" de multiplications de matrices, sa complexité

est environ de $7n^3 + 7n^2$, donc en $\mathcal{O}(n^3)$.

Algorithm 1: Multiplication matrices séquentiel

Data: A, B matrices, n taille des matrices

```
1  $M \leftarrow \text{Mat}[n][n]$ 
  for  $i = 0$  to  $n - 1$  do
    for  $j = 0$  to  $n - 1$  do
       $sum \leftarrow 0$ 
5      for  $k = 0$  to  $n - 1$  do
         $sum \leftarrow sum + A[i][k] \times B[k][j]$ 
       $M[i][j] \leftarrow sum$ 
8 return  $M$ 
```

Question 1 (/1)

Supposons que nous avons accès à un CPU mono-cœur qui a une fréquence d'1 GHz (1 milliard d'opérations sont effectuées à la seconde), pour 2 matrices de taille $n = 1\,024$, quelle serait la durée d'exécution, à la seconde près, de leur multiplication ? Même question pour une matrice de taille $n = 8\,192$?

Question 2 (/3)

En utilisant uniquement le multi-threading, expliquez comment l'Algorithme 1 peut être parallélisé. Quel serait le speed-up si vous utilisez 8 threads pour une taille de matrices de $n = 8\,192$? Y a-t-il un nombre idéal de threads à utiliser pour cette exécution ?

Question 3 (/10)

Vous avez à disposition :

- plusieurs ordinateurs, dont les cartes mères ont 1, 2 ou 4 sockets CPU, et 1 ou 2 sockets GPU
- plusieurs CPUs, comportant 1, 2, 4, 8 ou 32 cœurs de 3 GHz sur 64 bits, avec 2 threads par cœur
- plusieurs GPUs, comportant 1024, 2048, 4096 ou 8192 cœurs mono-thread de 1 GHz sur 64 bits
- les algorithmes suivants :
 - multiplication de matrices séquentielle de complexité $7n^{\log_2(7)} - 6n^2$
 - multiplication de matrices par bloc (avec des algorithmes de division en 4 sous-matrices et de fusion de 4 sous-matrices de complexité $7n \times \log_2(n)^2$ (Tiens donc, ça ressemble au tri fusion...))
 - compression sans perte de la taille des entiers (64 bits vers 32 ou 16 bits)

Avec ce que vous avez à disposition, donnez une méthode utilisant les différentes manières de paralléliser la multiplication de matrices pour diminuer le temps d'exécution. Quel temps cela prendrait-il pour 2 matrices de taille $n = 8\,192$? Déterminez le speed-up comparé à l'Algorithme 1. (Il n'y a pas une seule bonne réponse, c'est votre raisonnement qui compte)

2.2 Contrainte de stockage (/7)

Question 1 (/1)

Combien d'octets (1 octet = 8 bits) de stockage sont-ils nécessaires pour stocker une matrice de taille n ?

Question 2 (/1)

Si toutes les matrices étaient chargées en RAM, combien de giga octets de RAM faudrait-il pour exécuter l'Algorithme 1, pour des matrices de taille $n = 8\,192$?

Question 3 (/5)

Il n'est en réalité pas nécessaire de charger l'entièreté des matrices en RAM. Nous pouvons, par exemple, ne charger que la ligne i de la matrice A , la colonne j de la matrice B , et qu'un seul élément de M . En suivant ce raisonnement et en reprenant la méthode que vous avez donnée à la fin de la Section 2.1, calculez la RAM minimum nécessaire à chaque ordinateur, pour une multiplication de matrices de taille $n = 8\,192$.

2.3 Contrainte de communication (/7)

Question 1 (/1)

Supposons que nous utilisons un bus de données avec un débit de 240 Mo/s (avec une latence de 0 ms) entre la RAM et le stockage persistant. Quel est le temps nécessaire pour charger les matrices A et B de taille $n = 8\,192$ en RAM ? Même question pour charger puis décharger une ligne et une colonne en RAM.

Question 2 (/1)

Quelle méthode de la Question 1 donne un temps de chargement total plus petit ? En sachant que le chargement en RAM peut se faire parallèlement à l'exécution d'instructions, quelle méthode est la plus efficace ?

Question 3 (/1)

Pour transférer des données entre 2 ordinateurs, nous disposons d'une connexion de 1 Go/s, avec une latence de 200 ms. Calculez le temps que cela prendra de transmettre une matrice de taille $n = 2\,500$ d'un ordinateur à un autre.

Question 4 (/4)

Supposons que :

- chaque ordinateur peut utiliser parallèlement 4 connexions inter-ordinateurs d'un débit de 1 Go/s et de latence 100 ms
- les bus CPU-RAM ont un débit de 240 Mo/s avec une latence de 0 ms
- les bus GPU-RAM ont un débit de 2 Go/s avec 0 ms de latence de 0 ms
- chaque ordinateur possède 2 To de RAM (même chose pour les GPUs)

Pour la multiplication de 2 matrices de taille $n = 8\,192$, avec votre méthode définie à la fin de la Section 2.1, donnez approximativement le temps d'exécution de ces 2 matrices en prenant en compte les communications. Calculez à nouveau le speed-up.